# SIMATS ENGINEERING

## ASSESSMENT TOOL 1: Analytical Problem Solving

**COURSE NAME:** Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1504

---

**COURSE OUTCOME COVERED**

**CO2:** *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 50**
Question Count: 5

---

**Code of Conduct**

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

| Criteria | Problem Interpretation (2) | Analytical Reasoning (2.5) | Method Selection (2) | Solution Quality (2) | Presentation of Analysis (1.5) | TOTAL (10) |
|---|---|---|---|---|---|---|
| **Weightage** | **20%** | **25%** | **20%** | **20%** | **15%** | **100%** |
| Q1 | | | | | | |
| Q2 | | | | | | |
| Q3 | | | | | | |
| Q4 | | | | | | |
| Q5 | | | | | | |

Signature of the Faculty Total Marks Awarded:

**Analytical Problem Solving**

1. A cloud datacenter host has 64 CPU cores and supports CPU overcommitment at a ratio of 2.5:1. Each virtual machine (VM) requires 4 vCPUs. Calculate the maximum number of VMs that can be deployed on this host. If actual CPU utilization reaches 85%, analyze whether further VM allocation is advisable.
2. A virtualized storage system provides 20 TB of physical storage with thin provisioning at a ratio of 3:1. Each VM is allocated 500 GB of virtual disk space. Calculate the maximum number of VMs that can be provisioned.
3. Evaluate the security implications of multi-tenancy in cloud datacenters. Analyze how virtualization can both enable and weaken isolation, and recommend mechanisms to strengthen tenant separation.
4. Analyze the role of Service-Oriented Architecture (SOA) in enabling interoperability across heterogeneous cloud platforms. Identify key challenges such as service latency, security, and version control, and propose solutions.
5. A cloud system implementing presence services faces scalability issues during peak usage. Analyze the architectural bottlenecks and recommend design improvements to ensure real-time updates and efficient resource utilization.

Answer:

Q1. CPU Overcommitment Calculation

Given

- Physical CPU cores = 64
- Overcommitment ratio = 2.5: 1
- Each VM requires = 4 vCPUs
- Actual CPU utilization = 85%

Step 1: Calculate maximum vCPUs

$$64 \times 2.5 = 160 \text{ vCPUs}$$

Step 2: Calculate maximum number of VMs

$$\frac{160}{4} = 40 \text{ VMs}$$

Step 3: Analyze utilization

- Current utilization = 85%
- Safe operational range is usually 70–80%.
- 85% is already high, leaving little headroom for workload spikes.

Conclusion

- Maximum deployable VMs = 40
- Further VM allocation is not recommended because CPU utilization is already 85%, which may cause contention, increased latency, and reduced performance during peak load.

Q2. Thin Provisioning Storage Calculation

Given

- Physical storage = 20 TB
- Thin provisioning ratio = 3: 1
- Each VM allocated storage = 500 GB

Step 1: Convert TB to GB

$$20 \times 1024 = 20{,}480 \text{ GB}$$

Step 2: Calculate maximum logical storage

$$20{,}480 \times 3 = 61{,}440 \text{ GB}$$

Step 3: Calculate maximum number of VMs

$$\frac{61{,}440}{500} = 122.88$$

Final Answer

- Maximum VMs = 122 VMs
- The remaining 0.88 VM cannot be deployed because a VM requires a full 500 GB allocation.

Q3. Security Implications of Multi-Tenancy

How virtualization strengthens isolation

- Hypervisors create separate VM environments.
- Memory, CPU, and storage are logically isolated.
- A failure in one VM usually does not affect another VM.

How virtualization can weaken isolation

- Hypervisor vulnerabilities may allow VM escape attacks.
- Shared hardware components can expose side-channel attacks.
- Misconfigured virtual networks may permit unauthorized access.

Recommended mechanisms

- Strong hypervisor security: Regular patching and hardened configurations.
- Network segmentation: Use VLANs, VPCs, and micro-segmentation.
- Access control: Implement IAM, MFA, and least-privilege policies.
- Encryption: Encrypt data at rest and in transit.
- Monitoring: Use SIEM, intrusion detection, and audit logging.

Conclusion

Virtualization is the foundation of multi-tenancy, but effective tenant separation requires hypervisor hardening, network isolation, encryption, and continuous monitoring.

Q4. SOA Interoperability Analysis

Role of SOA

SOA enables different applications to communicate through standardized service interfaces, making systems developed in different languages and platforms interoperable.

Interoperability across cloud platforms

- Services communicate using REST, SOAP, HTTP, XML, and JSON.
- Applications can integrate across AWS, Azure, Google Cloud, and on-premise systems.

Challenges

| Challenge | Impact |
|---|---|
| Service latency | Slow response between distributed services |
| Security | Unauthorized access and data exposure |
| Version control | Breaking changes between service versions |
| Service discovery | Difficulty locating available services |

Solutions

- API Gateway for centralized routing and security.
- OAuth 2.0 and JWT for authentication.
- Semantic versioning (v1, v2 APIs).
- Service registry such as Consul or Eureka.

Conclusion

SOA improves interoperability and reusability, but large-scale cloud systems must address latency, security, and version management to maintain reliable service communication.

Q5. Presence Service Scalability Analysis

Introduction

A presence service continuously tracks whether users are online, offline, busy, or away. Applications such as WhatsApp, Microsoft Teams, Discord, and Google Meet rely on presence services to display user status in real time. During peak hours, millions of users send status updates simultaneously, creating scalability challenges.

Architecture Bottlenecks

1. Single Application Server

All requests are handled by one server, creating a performance bottleneck and a single point of failure.

2. Centralized Database

A single database receives continuous read and write operations, increasing latency and reducing throughput.

3. Synchronous Communication

Every status update waits for immediate processing before responding, slowing the system.

4. High Network Traffic

Millions of clients repeatedly send requests, consuming bandwidth and increasing response time.

5. Resource Contention

CPU, memory, and storage become overloaded during peak traffic, causing delays.

6. Lack of Load Balancing

Uneven request distribution overloads some servers while others remain underutilized.

Recommended Design Improvements

1. Horizontal Scaling

Deploy multiple application servers behind a load balancer so traffic is evenly distributed.

2. Auto Scaling

Automatically add new servers during high traffic and remove them during low traffic, reducing cost while maintaining performance.

3. Load Balancer

Distribute user requests equally across available servers to prevent overload.

4. Event-Driven Architecture

Use message queues such as Kafka or RabbitMQ to process updates asynchronously without blocking users.

5. In-Memory Cache

Store active presence information in Redis or Memcached for fast retrieval.

6. Distributed Database

Use database replication, partitioning (sharding), or NoSQL databases like Cassandra or MongoDB for higher scalability.

7. WebSockets

Use WebSocket connections instead of repeated HTTP polling for instant status updates with lower bandwidth usage.

8. Monitoring and Resource Optimization

Monitor CPU, memory, storage, and network utilization continuously. Configure alerts to detect overload conditions early

Example

During a cricket World Cup final, millions of users open WhatsApp simultaneously. Without autoscaling and load balancing, servers become overloaded and online status updates are delayed. With horizontal scaling, Redis caching, WebSockets, and distributed databases, the system can process millions of updates in real time while maintaining low latency and high availability.

Conclusion

Presence services require highly scalable architectures to handle large volumes of concurrent users. Combining horizontal scaling, autoscaling, load balancing, event-driven messaging, WebSockets, distributed databases, caching, and continuous monitoring ensures real-time communication, efficient resource utilization, reduced latency, and reliable service even during peak traffic.